Informe Técnico: Taller de Introducción a Redes Neuronales Artificiales

Curso de Inteligencia Artificial | Arquitectura Modular en Python (rna_core) | Propagación Hacia Adelante y Diseño

1. Una neurona que toma decisiones (Caso Alquiler de Apartamentos)

Modelo de decisión: x1 = precio / 8000, x2 = área / 80, z = -2*x1 + 3*x2 - 0.5, y_hat = sigma(z) = 1 / (1 + e^(-z)). La regla de decisión asigna clase c_hat = 1 ('se alquila') si y_hat >= tau, y 0 en caso contrario.

1.1. Tabla de cálculos (mínimo 4 decimales) y comparación con umbral tau = 0.8

Caso	Precio	Área	x1	x2	z	y_hat = sigma(z)	c_hat (tau=0.5)	c_hat (tau=0.8)
A	4000	60	0.5000	0.7500	+0.7500	0.6792	1 (Se alquila)	0 (No alquila)
B	6000	40	0.7500	0.5000	-0.5000	0.3775	0 (No alquila)	0 (No alquila)
C	2000	20	0.2500	0.2500	-0.2500	0.4378	0 (No alquila)	0 (No alquila)
D	7000	80	0.8750	1.0000	+0.7500	0.6792	1 (Se alquila)	0 (No alquila)

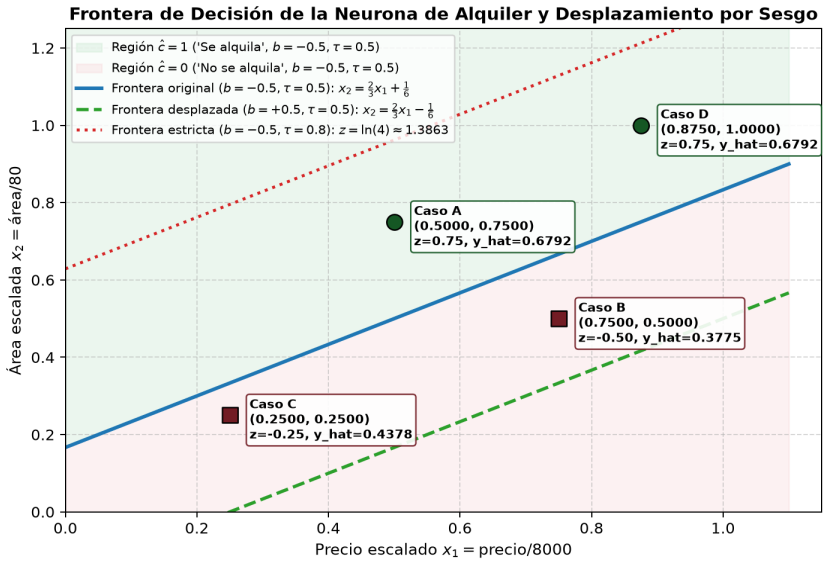
Análisis con tau = 0.8 (Punto 2): Al elevar el umbral de decisión a tau = 0.8, **cambian las decisiones discretas de los casos A y D** (pasan de clase 1 a clase 0, pues su puntuación continua es 0.6792 < 0.8). Los casos B y C permanecen en 0. **Las salidas continuas y_hat NO cambian en absoluto** porque dependen exclusivamente de las entradas, pesos y sesgo (activación y decisión son operaciones separadas).

Significado del signo de cada peso (Punto 3): El peso **w1 = -2 (negativo)** sobre el precio escalado indica una relación inversa: a mayor precio del apartamento, disminuye la suma ponderada z y se reduce la propensión a alquilar. El peso **w2 = +3 (positivo)** sobre el área escalada indica una relación directa: a mayor área en m², crece la suma ponderada z y aumenta la probabilidad de alquilar.

Despeje de la frontera y efecto del sesgo (Puntos 4 y 5): Para tau = 0.5, sigma(z) = 0.5 equivale a z = 0:

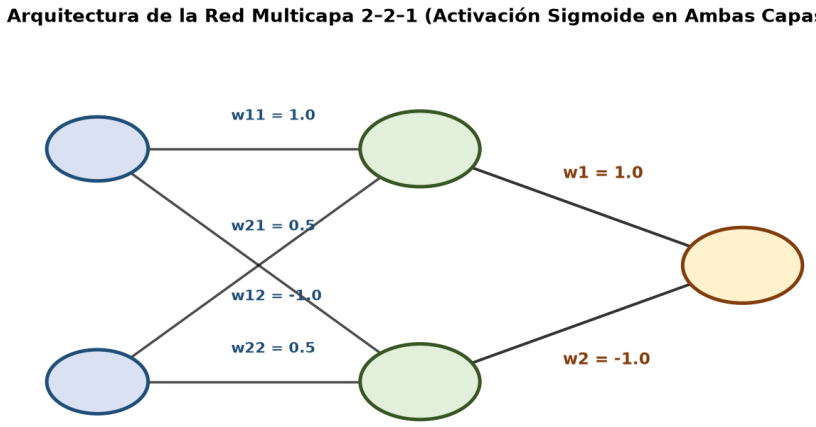
```
Con b = -0.5: -2*x1 + 3*x2 - 0.5 = 0 ==> x2 = (2/3)*x1 + 1/6 (intersección eje x2 en +0.1667)
Con b = +0.5: -2*x1 + 3*x2 + 0.5 = 0 ==> x2 = (2/3)*x1 - 1/6 (intersección eje x2 en -0.1667)
```

Cambiar el sesgo de b = -0.5 a b = +0.5 conserva intacta la pendiente de la recta (m = 2/3), pero **desplaza paralelamente la frontera hacia abajo** en 1/3 de unidad sobre el eje x2, ampliando la región de aceptación (hace que la neurona sea más permisiva para alquilar, pasando también a B y C a clase 1).


Frontera de Decisión de la Neurona de Alquiler y Desplazamiento por Sesgo
Región $\hat{c}=1$ ('Se alquila', $b=-0.5$, $\tau=0.5$)
Región $\hat{c}=0$ ('No se alquila', $b=-0.5$, $\tau=0.5$)
Frontera original ($b=-0.5$, $\tau=0.5$): $x_2=\frac{2}{3}x_1+\frac{1}{6}$
Frontera desplazada ($b=+0.5$, $\tau=0.5$): $x_2=\frac{2}{3}x_1-\frac{1}{6}$
Frontera estricta ($b=-0.5$, $\tau=0.8$): $z=\ln(4)\approx 1.3863$
Caso D
(0.8750, 1.0000)
z=0.75, y_hat=0.6792
Caso A
(0.5000, 0.7500)
z=0.75, y_hat=0.6792
Caso B
(0.7500, 0.5000)
z=-0.50, y_hat=0.3775
Caso C
(0.2500, 0.2500)
z=-0.25, y_hat=0.4378
Área escalada x_2 = área/80
Precio escalado x_1 = precio/8000
0.0
0.2
0.4
0.6
0.8
1.0
1.2

2. Propagación Multicapa (Red 2–2–1)

Se evalúa la red 2–2–1 con entrada x = [1, 0.5]^T y etiqueta real y = 1. Parámetros: W^(1) = [[1, -1], [0.5, 0.5]], b^(1) = [0, 0]^T, W^(2) = [1, -1], b^(2) = 0.



2.1. Cálculo paso a paso y comparación entre b^(2) = 0 y b^(2) = 1 (Puntos 2 a 5)

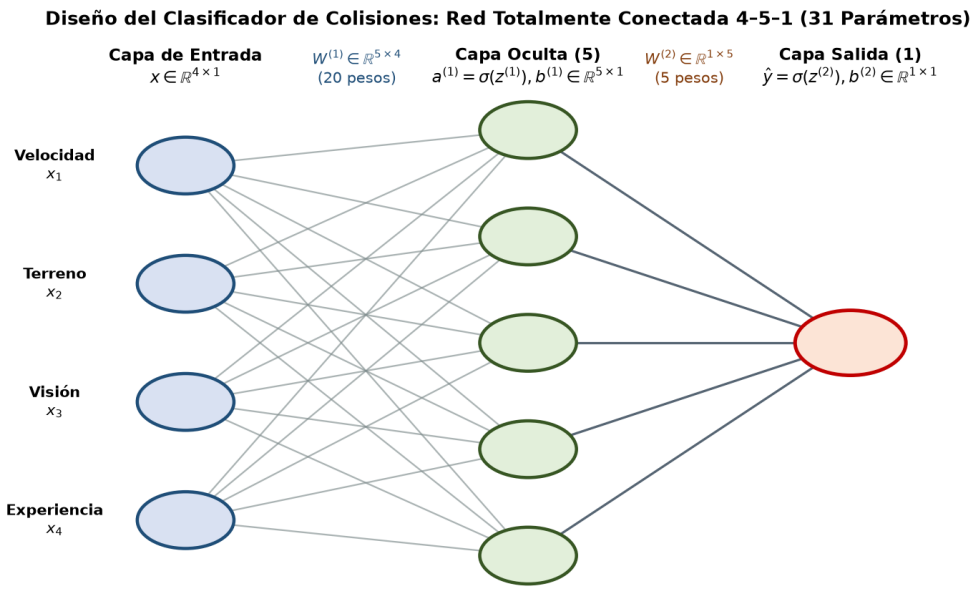
```
z1^(1) = 1*(1.0) + (-1)*(0.5) + 0 = 0.5000 ==> a1^(1) = sigma(0.5000) = 0.622459
z2^(1) = 0.5*(1.0) + 0.5*(0.5) + 0 = 0.7500 ==> a2^(1) = sigma(0.7500) = 0.679179
```

Configuración	z^(2) = a1 - a2 + b^(2)	y_hat = sigma(z^(2))	Clase (tau=0.5)	Etiqueta y	Pérdida BCE (L)
Original (b^(2) = 0)	-0.056719	0.485824	0 (Error)	1	0.721909
Modificada (b^(2) = 1)	+0.943281	0.719762	1 (Acierto)	1	0.328835

Conteo de parámetros (Punto 6): En una arquitectura 2–2–1 totalmente conectada hay: Pesos = (2x2) + (2x1) = 4 + 2 = **6 pesos**; Sesgos = 2 + 1 = **3 sesgos**; Total = **9 parámetros entrenables**.

Eliminación de la activación oculta y colapso afín (Punto 7): Si se suprime la sigmoide oculta, z^(2) = W^(2)(W^(1)x + b^(1)) + b^(2) = (W^(2)W^(1))x + (W^(2)b^(1) + b^(2)). Calculando el producto matricial: W_ef = [1, -1] @ [[1, -1], [0.5, 0.5]] = [0.5, -1.5]. Así, toda la red multicapa colapsa en una única transformación lineal-afín equivalente a una sola neurona: **z^(2) = 0.5*x1 - 1.5*x2 + b^(2)** (que para x=(1, 0.5)^T vale -0.25 con b^(2)=0). La activación no lineal es indispensable para romper la linealidad y permitir aproximar fronteras curvas o regiones no conexas (como XOR).

3. Ficha de Diseño de un Clasificador de Colisiones (Red 4–5–1)



Dimensiones con vectores columna (Punto 2): x in R^(4x1), W^(1) in R^(5x4), b^(1) in R^(5x1), z^(1), a^(1) in R^(5x1), W^(2) in R^(1x5), b^(2) in R^(1x1), z^(2), y_hat in R^(1x1). Para un lote de N columnas: X in R^(4xN), P in R^(1xN).

Arquitectura (d–h–k)	Fórmula Pesos (d*h + h*k)	Pesos	Sesgos (h + k)	Parámetros Totales
4–3–1	4(3) + 3(1) = 12 + 3	15	3 + 1 = 4	19
4–5–1	4(5) + 5(1) = 20 + 5	25	5 + 1 = 6	31
4–8–1	4(8) + 8(1) = 32 + 8	40	8 + 1 = 9	49

¿Por qué más neuronas no garantizan mejores predicciones? (Punto 4): Incrementar h aumenta la capacidad de representación, pero con pocos datos induce **sobreajuste (overfitting)**: la red memoriza el ruido del conjunto de entrenamiento y pierde generalización.

Flujo de extremo a extremo (Punto 5): [Recepción de telemetría y verificación de unidades (km/h, escala 0-10, grados, km)] --> [Imputación/rechazo de valores faltantes y validación de dominio] --> [Escalamiento Min-Max x' = x/limites] --> [Propagación matricial Z1 -> A1 -> Z2 -> P] --> [Umbralización P >= tau] --> [Alerta de Colisión y puntuación de riesgo].

Escalamiento de 150 km/h (Punto 6): Con máximo didáctico de 120 km/h, escalar 150 km/h produce **x'_1 = 150 / 120 = 1.25**. Justificación: **Conservar (1.25)** es preferible si 150 km/h es una velocidad físicamente real medida por el sensor, pues preserva la severidad del exceso de velocidad; **Recortar (clipping a 1.0)** evita extrapolación fuera del dominio entrenado pero pierde sensibilidad entre 120 y 150 km/h; **Rechazar** solo aplica si supera límites físicos de falla de sensor (ej. > 350 km/h o valores negativos).

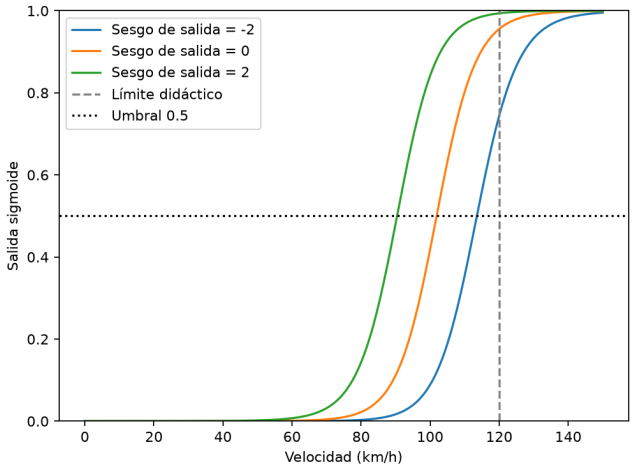
Partición para 1000 registros reales y escalador sin fuga de datos (Puntos 7 y 8): División estratificada **70% Entrenamiento (700)** para optimizar pesos y ajustar el escalador; **15% Validación (150)** para elegir arquitectura (h) y umbral (tau); **15% Prueba (150)** para estimar desempeño final sin sesgo optimista. Cuando los mínimos/máximos se estiman empíricamente de los datos, el escalador debe ajustarse **únicamente con entrenamiento** para evitar *data leakage* (fuga de información estadística del test).

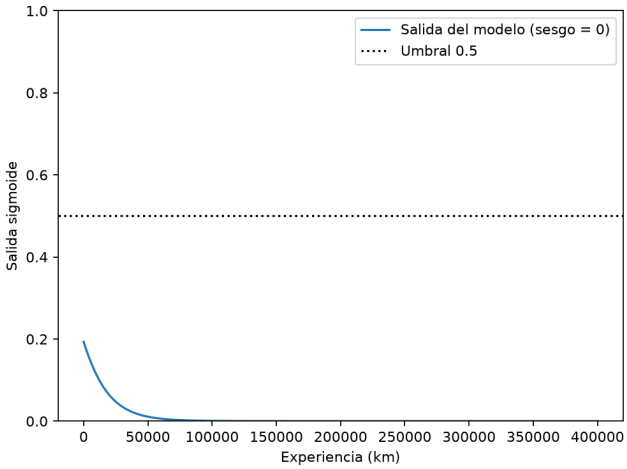
Información faltante en Terreno y Visión (Punto 9): En *Terreno (0-10)* falta definir la polaridad y protocolo de medición (¿0 es asfalto seco óptimo y 10 hielo/trocha peligrosa, o al revés? En los datos, valores bajos como 1, 2, 3 se asocian a colisión). En *Visión (0-360°)* falta especificar si mide el campo visual libre de obstáculos, visibilidad atmosférica o ángulo de cámaras/sensores.

4. Implementación en Python, Verificación y Experimentos

Al ejecutar **python main.py** se verifican las dimensiones matriciales: X (4, 7), W1 (5, 4), B1 (5, 1), A1 (5, 7), W2 (1, 5), B2 (1, 1) y P (1, 7). La propagación por lote coincide exactamente con la propagación individual (np.allclose = True). La exactitud didáctica es **1.0 (100%)** y la pérdida media de entropía cruzada es **0.00074555**.

Registro	Velocidad	Terreno	Visión	Experiencia	Etiqueta y	Puntuación P	Clase (0.3 / 0.5 / 0.8)	Acierto
1	65	5	180	80,000	0	0.002046	0 / 0 / 0	True
2	120	1	72	110,000	1	0.999962	1 / 1 / 1	True
3	8	6	288	50,000	0	0.000008	0 / 0 / 0	True
4	50	2	324	1,600	1	0.999182	1 / 1 / 1	True
5	25	9	36	160,000	0	0.000001	0 / 0 / 0	True
6	80	3	120	6,000	1	0.997698	1 / 1 / 1	True
7	40	3	360	400,000	0	0.000002	0 / 0 / 0	True





Interpretación de los experimentos: (1) En la curva de *Velocidad*, al aumentar la velocidad de 0 a 150 km/h (manteniendo terreno=5, visión=180, experiencia=80,000), la probabilidad predicha de colisión sube de forma monótona siguiendo una transición sigmoidal. Un sesgo de salida b2 = +2 desplaza la curva hacia la izquierda (alerta a menor velocidad, ~82 km/h), b2 = 0 cruza el umbral 0.5 cerca de 101 km/h, y b2 = -2 exige mayor velocidad (~119 km/h) para activar la alerta. (2) En la curva de *Experiencia*, a muy baja experiencia (< 25,000 km) el riesgo de colisión es alto, y cae drásticamente hacia cero a medida que el conductor acumula experiencia.

5. Análisis, Discusión y Reto Opcional XOR

1. ¿Qué cambia durante una predicción y qué permanece fijo? Permanecen fijos la arquitectura (capas, neuronas, funciones de activación), los pesos (W1, W2), los sesgos (b1, b2) y los límites del escalador. Cambian el vector de entrada x, las sumas ponderadas (z1, z2), las activaciones ocultas (a1) y la predicción final (y_hat).

2. ¿Qué diferencia hay entre diseñar y entrenar la red? Diseñar define la estructura e hiperparámetros antes del aprendizaje (entradas, número de capas/neuronas, activaciones, escalamiento). Entrenar es el proceso numérico de optimización (ej. descenso por gradiente y backpropagation) que ajusta los valores de los pesos y sesgos minimizando la pérdida sobre datos etiquetados.

3. Si se clasifican bien los 7 registros, ¿qué concluye y cómo evalúa generalización? Solo se concluye que el modelo tiene capacidad suficiente para separar esos 7 ejemplos puntuales. No demuestra validez estadística. La generalización se evalúa con un conjunto de prueba independiente, amplio y representativo (o validación cruzada) nunca visto durante el ajuste.

4. ¿Por qué una salida de 0.99 no demuestra probabilidad real del 99%? Porque la sigmoide comprime cualquier valor z grande al intervalo (0,1), pero una puntuación de confianza del modelo no es una probabilidad frecuentista real a menos que el modelo haya sido entrenado con datos representativos y verificado mediante curvas de calibración (ej. Brier score / Platt scaling).

5. ¿Qué pasaría si intercambiara las columnas de velocidad y experiencia? El escalador dividiría la velocidad entre 400,000 y la experiencia entre 120 (generando valores enormes de hasta 3,333), y las columnas de W1 multiplicarían características equivocadas. Esto destruye por completo el significado semántico y las predicciones del modelo (feature misalignment).

6. ¿Por qué variar una entrada describe al modelo pero no demuestra causalidad? Porque las curvas de sensibilidad muestran únicamente cómo responde la función matemática aprendida f_theta(x) ante cambios en una variable (asociación interna del modelo), no el mecanismo físico o causal del mundo real, el cual puede tener variables de confusión no medidas.

7. Distinga falsos positivos (FP) y falsos negativos (FN) y su efecto en el umbral. FP: predecir colisión cuando no ocurre (falsa alarma). FN: predecir ausencia de colisión cuando sí ocurre el accidente. En seguridad vial un FN tiene consecuencias catastróficas (pérdida de vidas), por lo que conviene reducir el umbral tau (ej. a 0.3) para maximizar la sensibilidad (recall) aunque aumenten moderadamente las falsas alarmas.

5.1. Reto Opcional: Compuerta XOR con Escalón de Heaviside

(x1, x2)	z_h1 (x1+x2-0.5)	h1 = H(z_h1)	z_h2 (x1+x2-1.5)	h2 = H(z_h2)	z_out (h1-2h2-0.5)	y_hat = H(z_out)	XOR Real
(0, 0)	-0.5	0	-1.5	0	-0.5	0	0
(0, 1)	+0.5	1	-0.5	0	+0.5	1	1
(1, 0)	+0.5	1	-0.5	0	+0.5	1	1
(1, 1)	+1.5	1	+0.5	1	-1.5	0	0

Dificultad del escalón H(z) para entrenar por gradientes: La función escalón H(z) es discontinua en z = 0 (donde su derivada no existe) y es constante en todo z != 0, por lo que su derivada es **dH/dz = 0 en casi todas partes**. Al aplicar la regla de la cadena en *backpropagation*, los gradientes se multiplican por cero, anulando la señal de actualización de los pesos e impidiendo que el descenso por gradiente aprenda.

